Convolutional Neural Networks

Tim Book

GENERAL ASSEMBLY

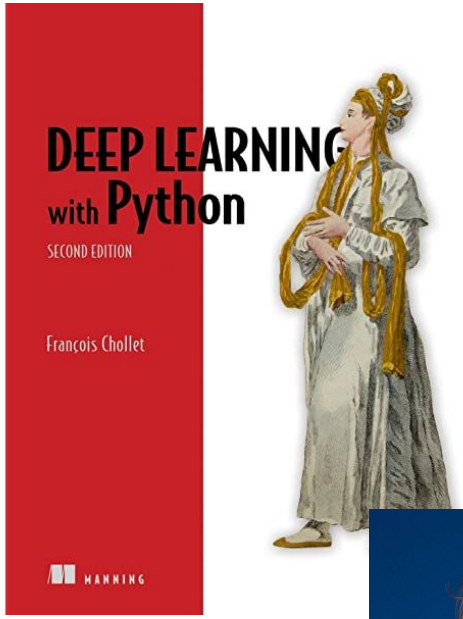
Agenda

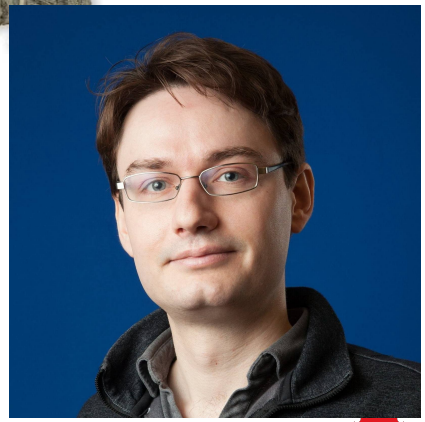
- What is computer vision, and what are the kinds of computer vision problems?
- How do CNNs work?
- What is the “convolution” operation?
- Python!

 © 2022 General Assembly

Citation

To give credit where credit is due, many of the examples and images used in this lesson are from **Deep Learning with Python** by **François Chollet**, the author of the Keras library!





What is Computer Vision?

Computer vision is the field of computer science that deals with image data. It's been basically entirely dominated by convolutional deep learning in the last few years, to the point where pretty much all computer vision research is now also deep learning research.

What is Computer Vision?

Computer vision problems typically fall into one of these 3 categories:

- Image classification
  - Is this a picture of a cat or a dog?
- Image segmentation
  - Divide up an image into its constituents at the pixel-level
  - Does this pixel belong to the cat or the background?
- Object detection
  - Draw rectangles (**bounding boxes**) around the object of interest in an image.

 © 2022 General Assembly

What is Computer Vision?

Computer vision problems typically fall into one of these 3 categories:

- Image classification
  - Is this a picture of a cat or a dog?
- Image segmentation
  - Divide up an image into its constituents at the pixel-level
  - Does this pixel belong to the cat or the background?
- Object detection
  - Draw rectangles (**bounding boxes**) around the object of interest in an image.



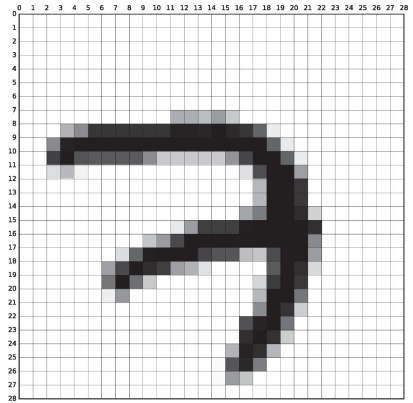
We'll be here today. It's the only one of these that's in scope for us.

 © 2022 General Assembly

We could start image classifying now...

We'll be using the classic MNIST handwritten digit data today. This time, in full 28x28 size!

We could take all 784 pixels that make up an image, lay them out, and use them in a dense neural network like we've already done.

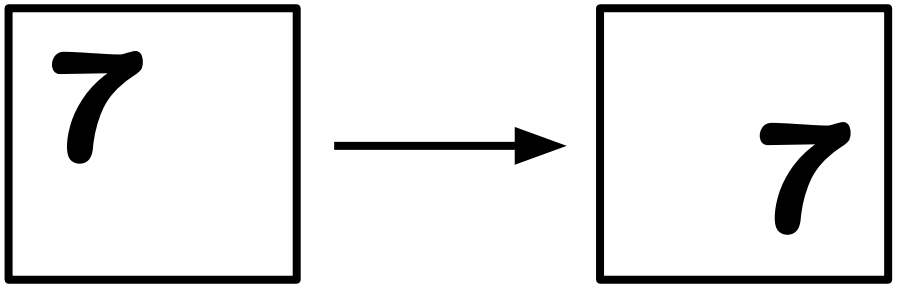


What's wrong with this?

Two major principles of image recognition

Principle #1: The visual world is fundamentally translation-invariant!

If we take an image and move (another word for *translate*) it a little bit, it still contains the same information. If we were to use a dense neural net (DNN), our pixel inputs would be totally different and our model might not recognize it!

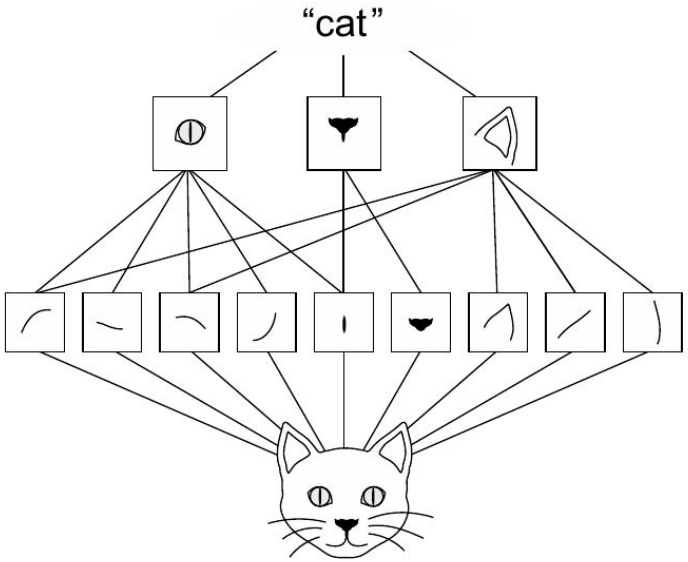


 © 2022 General Assembly

Two major principles of image recognition

Principle #2: The visual world is fundamentally spatially hierarchical!

The images and shapes we recognize are actually built up of smaller parts of shapes we recognize and assemble into a whole. DNNs have no way to understand this, but CNNs do!



 © 2022 General Assembly

CNNs: How do they work?

The Convolution Operation

The special operation that makes CNNs special is **convolution**. To do this, we'll have a special **2D convolution layer** in our models. To add a 2D conv layer, we're going to specify

1. How many convolution **filters** to generate and
2. How large these filters are (almost always they're 3x3)

 © 2022 General Assembly

So, what's a filter?

A **convolution filter** is a (usually 3x3) square matrix that we use to **convolve** input. These matrices are made up of **trainable weights**. The model figures out the best ones for us.

What does it mean to convolve something? I have a convolution demo in our other slide deck, so let's go there now!

(We’re in another slide deck right now…)

Max-Pooling

We generally follow 2D conv layers with **max-pool layers**. This involves cutting our image into disjoint 2x2 grids and downsampling to the largest value in that sub-grid. We do this for two reasons:

- To help the **spatial hierarchy** of the CNN - that is, make it so that the filters can “see” a broader subset of the image in later layers.
- Efficiency - each max-pool quarters the size of the image. Turns out images have a lot of unnecessary pixels!

 © 2022 General Assembly

The Max-Pool operation

7	8	3	0
3	0	1	9
-2	4	7	5
3	-3	1	8

 © 2022 General Assembly

The Max-Pool operation

7	8	3	0
3	0	1	9
-2	4	7	5
3	-3	1	8



7	8	3	0
3	0	1	9
-2	4	7	5
3	-3	1	8

The Max-Pool operation

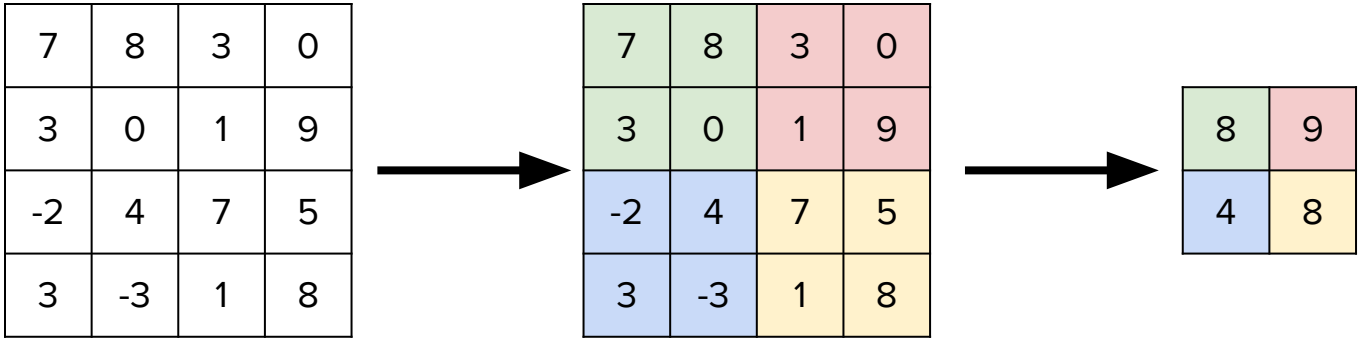
7	8	3	0
3	0	1	9
-2	4	7	5
3	-3	1	8



7	8	3	0
3	0	1	9
-2	4	7	5
3	-3	1	8

 © 2022 General Assembly

The Max-Pool operation



 © 2022 General Assembly

CNN Layering Intuition

What do these operations do to our data?

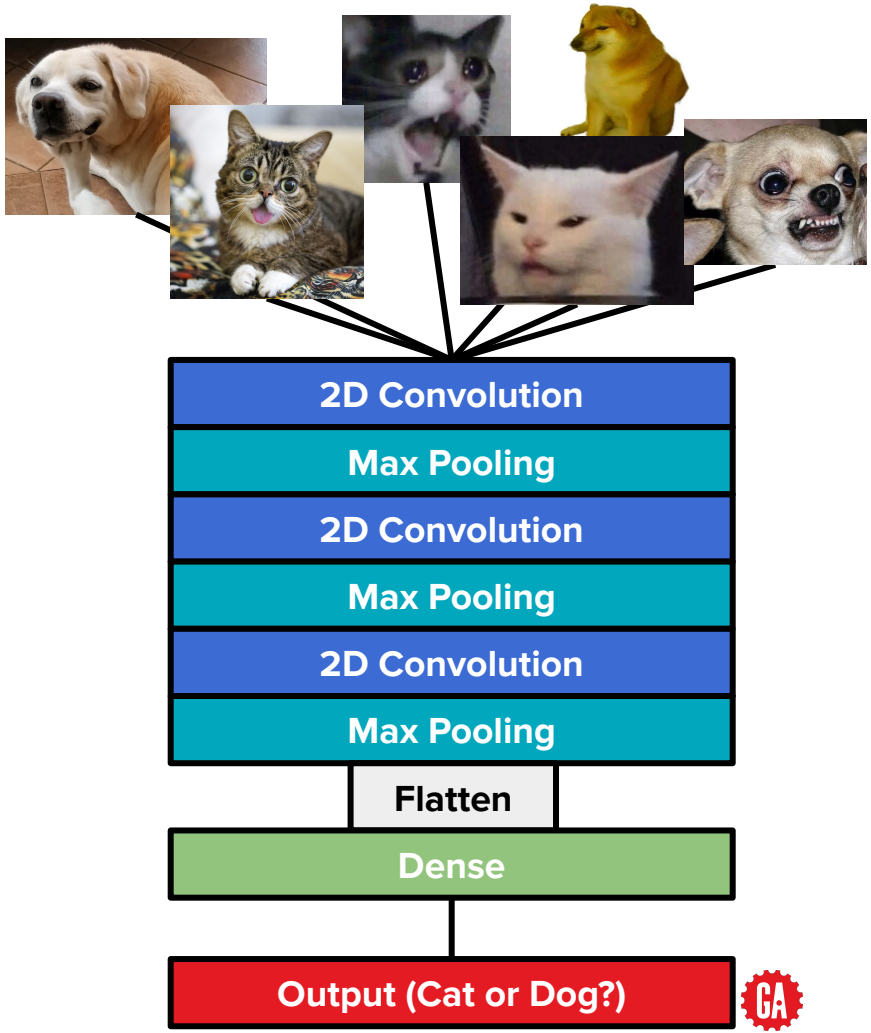
- Early convolution filters **learn vague shapes** in the input images.
- More filters = more vague shapes get learned! But there are only so many kinds of vague shapes - these are mostly just “edge detectors”.
- Later layers **learn more specific shapes**
- Since specific shapes are more specialized, we want more of them to learn to recognize more kinds of images

Intuition: The number of convolution filters should **increase** as you go deeper through the net.

A typical CNN

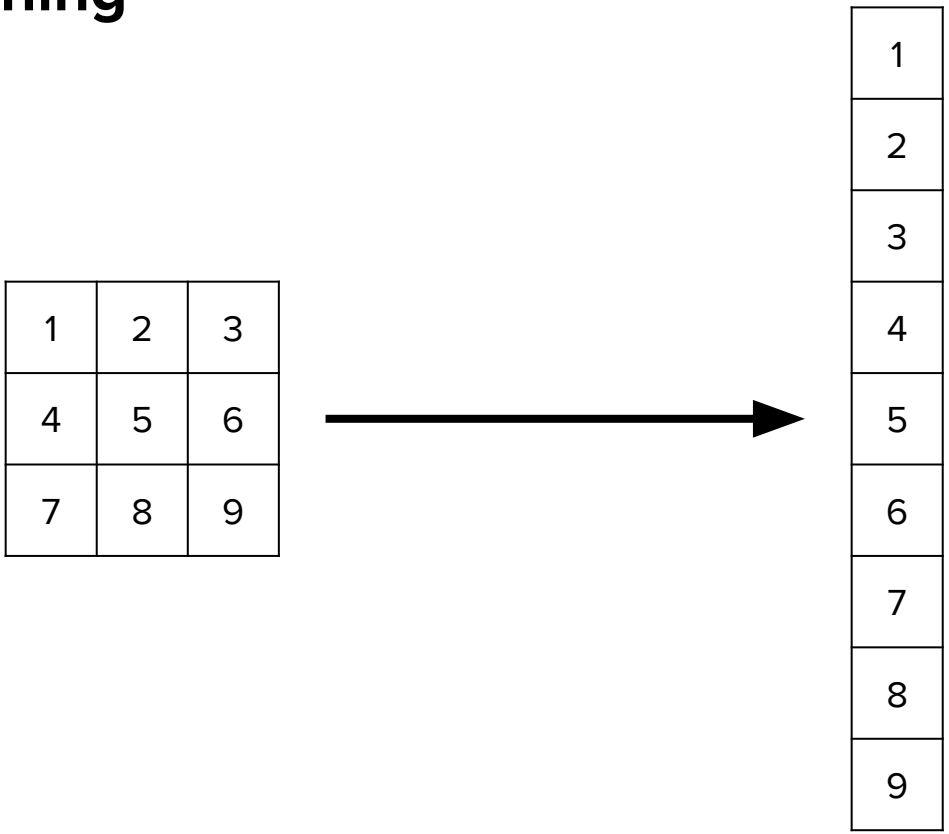
A typical CNN is made up of the following patterns:

1. Convolution blocks, made up of:
   a. A 2D conv layer
   b. A max-pool layer
2. A “flatten” layer that unrolls the convolved images
3. Dense layers (usually only one) to connect to the output
4. The output layer



 © 2022 General Assembly

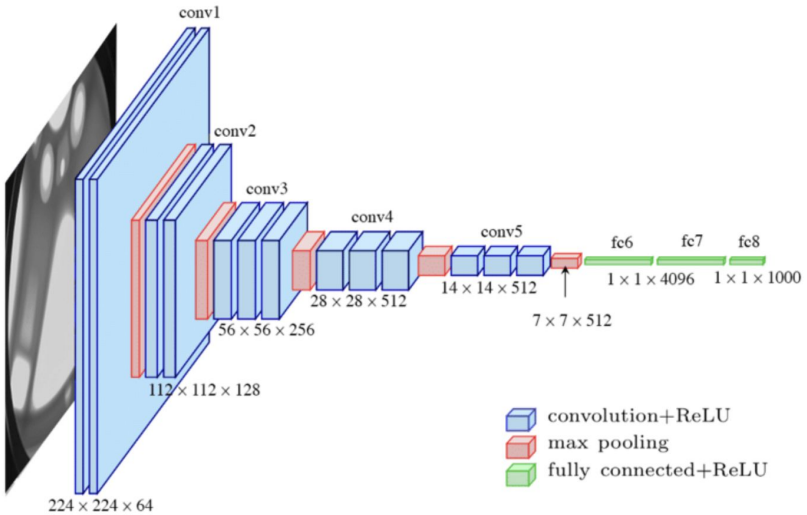
Aside: Flattening



 © 2022 General Assembly

A Specific Example CNN: VGG16

The **VGG16** model is one of many **pre-trained CNNs** available as part of Keras. It's a gigantic CNN trained on the famous **ImageNet** competition data to predict 1000 different output classes.



 © 2022 General Assembly

A Specific Example CNN: VGG16

It's actually possible to use pre-trained neural nets to engage in **transfer learning**.

Specifically, if we "freeze" the convolution weights from VGG16 and replace the dense nodes with new, trainable ones and fit this resulting amalgam on new data, we can take advantage of the generality of CNNs and fit great image-detection models on very little data!

A Specific Example CNN: VGG16

It's actually possible to use pre-trained neural nets to engage in **transfer learning**.

Specifically, if we "freeze" the convolution weights from VGG16 and replace the dense nodes with new, trainable ones and fit this resulting amalgam on new data, we can take advantage of the generality of CNNs and fit great image-detection models on very little data!

This is outside the scope of our class, but only barely! I highly encourage all students trying to fit CNNs for a capstone to try out transfer learning like this!

Let's try it out!



 © 2022 General Assembly